SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL – ASSIGNMENT 1

Course Code:	CSA17	**Course Title:**	Artificial Intelligence
CO Assessed:	CO1	**Assessment:**	Assignment 1 –Intelligent Agents and Uninformed Search
Weightage:	2% of CIA	**Duration / Marks:**	50 marks
CO1 Statement:	Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)		
Student Name:	Tejhash P	**Reg. No.:**	192512495
Section / Batch:	1	**Date:**	11/10/2026

Code of Conduct

I Tejhash P, 192512495 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Tejhash.P

Instructions

- Read each task carefully before starting. This is a take-home assignment completed over one week.
- Each task is a problem drawn from the prescribed textbooks (Russell and Norvig, Alpaydin, Manning et al.). Select the technique that fits, apply it step by step, and support every decision with evidence.
- Tasks are drawn from Intelligent Agents and Environments, Rationality, Problem Formulation, Uninformed Search Strategies (BFS, DFS, UCS) and Search with Partial Information.
- Every implemented task must be supported by: Source Code → Test Input → Expected Behaviour → Actual Output → Performance Metrics → Interpretation. Where a task says Predict, write the prediction before running the experiment; use fixed random seeds. Programming language: Python 3 (Jupyter / Colab).
- Show derivations, tables of operation counts, traces and graphs, and state the final answer of every task clearly.
- Marks will be awarded using the Assignment rubric: conceptual understanding, application of concepts, analytical thinking, organization & presentation, and completeness & accuracy.
- Upload your work in GCR and Github.

Reference Books Note

- T1 – Stuart Russell and Peter Norvig, Artificial Intelligence: A Modern Approach, 3rd Edition, PHI / Pearson Education (AIMA).
- Chapters used: Chapter 2 (Section 2.1–2.4: agents, rationality, PEAS, task environments, agent structure; the vacuum world of Figures 2.2 and 2.8); Chapter 3 (Section 3.1 problem formulation and the Romania road map of Figure 3.2; Section 3.4 breadth-first, uniform-cost, depth-first, depth-limited and iterative-deepening search; Figure 3.21; Exercises 3.9 and 3.15); Section 4.4 (searching with partial information, the sensorless vacuum world).
- Figure, table, section and exercise numbers in this assignment refer to the editions used for the problems: AIMA 3rd edition (T1). The road distances of the Romania map are reproduced from Figure 3.2.

Section / Topic	Focus	Task Nos.

1. Intelligent Agents and Problem Formulation	PEAS, rationality, task environments, agent structure, problem formulation (Romania agent, vacuum world)	**Task 1**
2. Uninformed Search: Analysis	Basic operation, complexity of BFS/DFS/UCS/IDS, best/average/worst case, asymptotics, recurrences	**Tasks 2–7**
3. Implementation, Experiments and Partial Information	Implementation and tests, experiments, theory vs observation, recommendation, sensorless problem and belief states	**Tasks 8–12**

Rubrics for Assignment

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	**25%**	Demonstrates complete and accurate understanding of concepts with strong clarity	Shows good understanding with minor gaps	Partial understanding with some errors	Lacks basic understanding of concepts
Application of Concepts	**25%**	Applies concepts correctly to problems with logical and appropriate solutions	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts correctly
Analytical Thinking	**20%**	Provides in-depth analysis with strong reasoning and justification	Shows logical analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Organization & Presentation	**15%**	Well-structured, neat, and logically organized with clear flow	Generally organized with minor issues	Some structure but lacks clarity	Poorly organized and difficult to follow
Completeness & Accuracy	**15%**	All questions attempted with accurate and error-free answers	Most answers correct with minor mistakes	Some answers incomplete or incorrect	Many answers missing or incorrect

Task / Criteria	Conceptual Understanding (25%)	Application of Concepts (25%)	Analytical Thinking (20%)	Organization & Presentation (15%)	Completeness & Accuracy (15%)	Sub. Total (Max)	Total %
Task 1 (5)						__ / 5	
Task 2 (2)						__ / 2	
Task 3 (6)						__ / 6	
Task 4 (4)						__ / 4	
Task 5 (3)						__ / 3	
Task 6 (3)						__ / 3	
Task 7 (4)						__ / 4	
Task 8 (4)						__ / 4	
Task 9 (3)						__ / 3	
Task 10 (4)						__ / 4	
Task 11 (4)						__ / 4	

Task 12 (8)						__ / 8	
Total (50)	__ / 12.5	__ / 12.5	__ / 10	__ / 7.5	__ / 7.5	__ / 50	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Scenario — The Romania Road-Trip Agent and the Vacuum World (Russell and Norvig, AIMA 3e, Chapters 2–4)

An agent is on holiday in Arad, Romania. Its flight leaves from Bucharest tomorrow, so it must decide how to get there (AIMA Section 3.1). The simplified road map of Figure 3.2 has 20 cities and 23 two-way roads with the following road distances in kilometres:

Road	km	Road	km
Oradea–Zerind	71	Craiova–Pitesti	138
Zerind–Arad	75	Rimnicu Vilcea–Pitesti	97
Arad–Timisoara	118	Pitesti–Bucharest	101
Arad–Sibiu	140	Fagaras–Bucharest	211
Oradea–Sibiu	151	Bucharest–Giurgiu	90
Sibiu–Fagaras	99	Bucharest–Urziceni	85
Sibiu–Rimnicu Vilcea	80	Urziceni–Hirsova	98
Timisoara–Lugoj	111	Hirsova–Eforie	86
Lugoj–Mehadia	70	Urziceni–Vaslui	142
Mehadia–Drobeta	75	Vaslui–Iasi	92
Drobeta–Craiova	120	Iasi–Neamt	87
Craiova–Rimnicu Vilcea	146		

As the map grows, the state space may grow:
20 → 2,000 → 200,000 → 20,000,000 cities
You are required to apply AI basics such as intelligent agents and uninformed search methods to solve the routing problem effectively: investigate, design and compare uninformed search strategies (breadth-first, depth-first and uniform-cost search) by which the agent finds a route between any two cities of the map.
Two further environments from the same book are used. The two-square vacuum world of AIMA Chapter 2 (Figure 2.2) is used for the agent analysis in Task 1 and for the Day-4 Design Challenge. The infinite binary-tree state space of AIMA Exercise 3.15 (state k has the two successors 2k and 2k + 1) is a scalable test space for Tasks 8–10.
Your solution must remain practical for both small and very large state spaces. Any preprocessing (for example, building an adjacency list or an explored set) must be included in your analysis.
Design Challenge (Day 4): The vacuum agent's location and dirt sensors fail, so it no longer receives any percept (AIMA Section 4.4). Re-evaluate your design under this changed requirement.

Task 1: Apply AI Basics: Analyze the Intelligent Agent, Formulate the Problem and Design the Uninformed Search Strategies (5 Marks)

CO1 Topic Mapping: Topic 1 – Introduction to Artificial Intelligence and AI Problems; Topic 2 – Foundations and History of Artificial Intelligence; Topic 3 – Intelligent Agents and Agent Environments; Topic 4 – Concept of Rationality and Rational Agents; Topic 5 – Nature of Task Environments; Topic 6 – Structure and Types of Intelligent Agents; Topic 7 – Problem-Solving Agents and Problem Formulation; Topic 8 – Searching for Solutions and Uninformed Search Strategies (BFS, DFS, UCS)
Before designing algorithms, analyze the agents and formulate the problem:
(a) Design the PEAS description of the Romania agent and of the vacuum cleaner (AIMA Section 2.1), and classify both environments (observable, single/multi-agent, deterministic, episodic/sequential, static/dynamic, discrete/continuous)
(b) Determine which view of AI (acting rationally, thinking rationally, acting humanly, thinking humanly) the Romania agent embodies, using its performance measure as evidence
(c) Evaluate the rationality of the vacuum agent of AIMA Figure 2.8: with the performance measure +1 for every clean square after each step and −1 for every move, compute the score over T = 8 steps of (i) that simple reflex agent and (ii) a reflex agent with state that performs NoOp when both squares are known to be clean, for each of the four initial dirt configurations (agent starts in square A; the agent acts first and the score is added after the action)

(d) Select the structure of the Romania agent (simple reflex, model-based reflex, goal-based, utility-based or learning) and evaluate the choice by computing the size of a table-driven design with one entry for every percept sequence of length 1 to 10 (a percept is the current city, 20 possible cities)
(e) Formulate the problem: states, initial state In(Arad), actions Go(city), transition model, goal test In(Bucharest) and path cost (road kilometres)
Then design and propose at least three meaningfully different uninformed search strategies for finding a route.
The strategies must be chosen from the uninformed family: breadth-first search, depth-first search, uniform-cost search and their depth-limited or iterative-deepening variants.
Your proposed approaches must include:
(a) At least one iterative approach (queue- or stack-based)
(b) At least one recursive approach
(c) A clear idea behind each approach
(d) Pseudocode or an appropriate flowchart for each approach
(e) A comparison criterion (expected nodes, memory or cost) for selecting each approach for investigation
(f) Any preprocessing required, such as adjacency-list construction or an explored set
You are expected to select the approaches independently. The appropriateness and diversity of the selected strategies will be assessed.
Evidence Expected: PEAS and environment classification, vacuum-agent scores, table-driven size, complete problem formulation, three distinct strategies with pseudocode and selection criteria.

Task 2: Determine the Basic Operation (2 Marks)

CO1 Topic Mapping: Topic 8 – Searching for Solutions and Uninformed Search Strategies (BFS, DFS, UCS)

For each proposed algorithm, determine the basic operation that predominantly determines its execution time on the Romania map.
Analyze:
(a) What the basic operation is (for example, node expansion, successor generation, goal test or frontier insertion)
(b) Where it occurs in the algorithm
(c) Why it is appropriate for efficiency analysis
(d) How the number of times it executes changes as the map grows from 20 to 20,000,000 cities
Do not simply name an operation. Relate it to the behaviour of the algorithm.
Evidence Expected: Correct determination of the basic operation for each algorithm, supported by operation counts on the Romania map.

Task 3: Derive the Time- and Space-Complexity Functions Mathematically (6 Marks)

CO1 Topic Mapping: Topic 8 – Searching for Solutions and Uninformed Search Strategies (BFS, DFS, UCS)

Derive the time and space complexity for each proposed approach using the parameters of AIMA Section 3.4: branching factor b, depth d of the shallowest goal, maximum depth m, optimal cost C* and minimum step cost ε.
Analyze:
(a) The number of nodes generated, derived by summing the nodes at each level (for example, $b + b^2 + \dots + b^d$ for breadth-first search)
(b) The expressions for time and for memory (frontier size) of breadth-first, uniform-cost, depth-first, depth-limited and iterative-deepening search
(c) Any preprocessing cost involved, and the overall cost when preprocessing and searching are considered together
(d) A numerical check on the Romania map: with an average branching factor $b = 2 \times 23 / 20$, $C^* = 418$ km for Arad to Bucharest and $\varepsilon = 70$ km, compute the uniform-cost bound $b^{(1+\lfloor C^*/\varepsilon \rfloor)}$ and compare it with the number of nodes your implementation actually expands

You must show the mathematical reasoning. Merely stating that an algorithm is O(b^d) will not receive full credit.
Evidence Expected: A clear derivation of time and space functions for each approach, the Romania numerical check and a comparison with measured expansions.

Task 4: Analyse Best, Average and Worst-Case Behaviour (4 Marks)

CO1 Topic Mapping: Topic 8 – Searching for Solutions and Uninformed Search Strategies (BFS, DFS, UCS)

Analyse each algorithm under different input conditions, using start–goal pairs of the Romania map and of the binary-tree space.
Analyze, wherever applicable:
(a) What input condition produces the best case (for example, the goal is a neighbour of the start, or lies on the first branch explored)
(b) What condition produces the average case (for example, a random pair of the 380 ordered city pairs)
(c) What condition produces the worst case (for example, goal at maximum depth, goal unreachable, a cheap-edge detour for uniform-cost search)
(d) How the number of nodes generated changes in each case, and why
Do not simply provide a table of complexity values. Derive the input conditions that lead to each case.
Evidence Expected: A best-, average- and worst-case analysis supported by mathematical reasoning and chosen city pairs.

Task 5: Perform Asymptotic Analysis (3 Marks)

CO1 Topic Mapping: Topic 8 – Searching for Solutions and Uninformed Search Strategies (BFS, DFS, UCS)

Express the growth of each algorithm's time and memory using O, Ω and Θ notation, and compare growth behaviour numerically.
(a) Express $N(BFS) = b + b^2 + \ldots + b^d$ and $N(IDS) = d \cdot b + (d-1)b^2 + \ldots + 1 \cdot b^d$ asymptotically and prove that both are $\Theta(b^d)$
(b) Evaluate both for $b = 10$ and $d = 5$ and compute the percentage by which iterative deepening exceeds breadth-first search (AIMA Section 3.4.5 reports 123,450 against 111,110 nodes; verify these values)
(c) Determine the value of b for which the iterative-deepening overhead falls below 10%
Evidence Expected: Correct asymptotic representation, the numerical verification and a meaningful comparison of growth behaviour.

Task 6: Formulate the Recurrence Relation (3 Marks)

CO1 Topic Mapping: Topic 8 – Searching for Solutions and Uninformed Search Strategies (BFS, DFS, UCS)

Formulate recurrences for the number of nodes generated.
(a) Let N(d) be the nodes generated by breadth-first search to depth d, D(l) the nodes generated by one depth-limited search with limit l, and I(d) the total for iterative deepening; formulate N, D and I as recurrences in d (or l)
(b) Identify the components of each recurrence (recursive part, work added per level) and the base cases
(c) Formulate a recurrence for the number of nodes a depth-first search stores, as a function of m and b
Evidence Expected: A correct recurrence with clear identification of its components and base case.

Task 7: Solve the Recurrence Relation (4 Marks)

CO1 Topic Mapping: Topic 8 – Searching for Solutions and Uninformed Search Strategies (BFS, DFS, UCS)

Solve the recurrences of Task 6.
(a) Derive closed forms for N(d), D(l) and I(d) by repeated substitution or summation
(b) Verify the closed forms against direct evaluation for $b = 10$, $d = 5$ and for $b = 2$, $d = 10$ (the binary-tree space)
(c) Interpret the result: for which b and d is the repeated work of iterative deepening acceptable?
Evidence Expected: A complete solution with the derivation of the method and interpretation of the result.

Task 8: Implement and Test the Algorithms (4 Marks)

CO1 Topic Mapping: Topic 7 – Problem-Solving Agents and Problem Formulation; Topic 8 – Searching for Solutions and Uninformed Search Strategies (BFS, DFS, UCS)

Implement your selected algorithms in Python 3 and test them on the Romania map, the binary-tree space and the river-crossing puzzle.

(a) Romania map: your uniform-cost search must find Arad–Sibiu–Rimnicu Vilcea–Pitesti–Bucharest (418 km), whereas breadth-first search must return the route with the fewest roads (Arad–Sibiu–Fagaras–Bucharest, 450 km)

(b) Binary-tree space with goal 11 (AIMA Exercise 3.15): your breadth-first search must visit states 1 to 11 in order; depth-limited search with limit 3 must visit 1, 2, 4, 8, 9, 5, 10, 11; iterative deepening must find the goal in its fourth iteration

(c) Missionaries and cannibals (AIMA Exercise 3.9): formulate the puzzle (state (m, c, boat)), run your search, and confirm the 11-crossing optimal solution and the number of legal and reachable states

(d) Test boundary conditions: start equals goal, unreachable goal, city with a single road, repeated states

Evidence Expected: Working code, test inputs, expected behaviour and actual outputs, with a reproducible testing procedure.

Task 9: Record Experimental Performance (3 Marks)

CO1 Topic Mapping: Topic 8 – Searching for Solutions and Uninformed Search Strategies (BFS, DFS, UCS)

Collect experimental data systematically. Predict the results before running the experiments.

(a) Binary-tree space: goals at depths 5, 7, 9, 11, 13, 15; record nodes generated, nodes expanded and peak frontier size for each algorithm

(b) Romania map: all 380 ordered city pairs; record average nodes expanded, average path cost and the share of pairs on which each algorithm returns the cheapest route

(c) Fixed random seeds, at least five repetitions for any randomised choice, and tables and graphs for all results

Evidence Expected: Systematically collected experimental data with clear tables and graphs.

Task 10: Compare Theory with Experimental Behaviour (4 Marks)

CO1 Topic Mapping: Topic 8 – Searching for Solutions and Uninformed Search Strategies (BFS, DFS, UCS)

Compare the Task 3–7 analysis with your Task 9 data.

(a) Fit the measured growth on the binary-tree space against b^d and compare with your predictions

(b) Locate any discrepancy (for example, depth-first search on the Romania map exploring fewer nodes than b^m) and determine its cause

(c) Compare the measured iterative-deepening overhead with your Task 5 value

Evidence Expected: A reasoned comparison between theoretical and observed performance.

Task 11: Recommend and Defend the Most Effective Final Design (4 Marks)

CO1 Topic Mapping: Topic 7 – Problem-Solving Agents and Problem Formulation; Topic 8 – Searching for Solutions and Uninformed Search Strategies (BFS, DFS, UCS)

Based on your mathematical analysis and experimental findings, recommend the uninformed search strategy that solves the routing problem most effectively for the Romania map and for a map of 20,000,000 cities.

(a) Decide the conditions on edge costs, memory and depth under which your recommendation changes (AIMA Figure 3.21 compares completeness, optimality, time and space)

(b) Evaluate your recommendation against at least one alternative using your own measurements

Evidence Expected: A recommendation derived from analysis and experiment.

DAY-4 DESIGN CHALLENGE

Task 12: Re-evaluate and Adapt the Design: Search with Partial Information (8 Marks)

CO1 Topic Mapping: Topic 9 – Search with Partial Information

Changed Requirement

In the two-square vacuum world the agent no longer senses its location or the dirt: it knows only that it may be in any of the 8 physical states, and it must still end with both squares clean (AIMA Section 4.4, the sensorless problem).

Re-evaluate your original design. You may RETAIN → MODIFY → or REDESIGN your original solution.

Note: this is search with partial information. The percept no longer identifies the state, so the agent must search in the space of belief states or use contingency planning with local sensing.

A. Impact Analysis – 2 Marks

(a) Determine which assumptions of the original design (fully observable, known start state) are affected, and compute the number of physical states (n·2^n for n squares) and possible belief states (2 to the power of that number) for n = 2 and n = 3

(b) Construct the belief state after the action Right and after Right, Suck, to show how the agent's knowledge now forms part of the state

B. Evaluation of the Original Design – 2 Marks

(a) Test whether your original algorithm can still be applied by running it on one belief-state example, and determine what a "state" becomes

(b) Compute how many of the 256 belief states are reachable from the initial belief state (AIMA reports 12) and judge whether the original design is acceptable

C. Adaptation or Redesign – 2 Marks

(a) Design the revised strategy (search over belief states, or search with a local dirt sensor and a conditional plan) and evaluate the effect on complexity; confirm that your search finds a plan equivalent to [Right, Suck, Left, Suck]

(b) If you retain the original solution, evaluate it against the new condition with a worked example

D. Evidence-Based Defence – 2 Marks

(a) Defend your decision using mathematical analysis and a small experiment (belief states generated and plan length for n = 2, 3 and 4 squares, with and without a local sensor), with code and output as proof

Answers – Assignment 1: Intelligent Agents and Uninformed Search (CSA17, CO1)

Scenario: the Romania road-trip agent and the two-square vacuum world (Russell and Norvig, AIMA 3e). All programs are Python 3 (runs in Jupyter / Colab); every number below was produced by running the code in this report. Fixed random seed: `random.seed(42)`. Neighbours are always expanded in alphabetical order, so every result is reproducible.

Task 1: Analyse the agent, formulate the problem, design the strategies (5 marks)

(a) PEAS description and environment classification

	Romania road-trip agent	Vacuum-cleaner agent
Performance measure	Reach Bucharest before the flight, with minimum total road distance (km); fewer expansions is a secondary efficiency goal	+1 for every clean square after each step, −1 for every Left or Right move (T = steps)
Environment	Road map of 20 cities and 23 two-way roads with distances (AIMA Fig. 3.2)	Two squares A and B, each Clean or Dirty; dirt does not reappear
Actuators	Go(city): drive along a road to an adjacent city	Suck, Left, Right, NoOp
Sensors	Current city (signposts / GPS) and the road map with distances	Location sensor and a dirt sensor for the current square only

Property	Romania environment	Vacuum world
Observable	Fully: the map and current city are known	Partially: sees only the dirt of its own square (location is known)
Agents	Single agent	Single agent
Deterministic?	Deterministic: Go(y) always ends in In(y)	Deterministic: actions have certain results
Episodic / sequential	Sequential: each road affects the remaining trip cost	Sequential: moves and cleaning affect future scores
Static / dynamic	Static: the map does not change while planning	Static (dirt never reappears, no other agent)
Discrete / continuous	Discrete: 20 cities, finite roads	Discrete: 2 squares, 4 actions, discrete time

(b) Which view of AI?

The Romania agent embodies **acting rationally**. Its performance measure (arrive in Bucharest at the least road cost) defines what a "right" action is, and the agent simply picks actions expected to maximise that measure. It is not trying to imitate a human driver (acting humanly), to copy human reasoning (thinking humanly), or to derive a route by logical inference only (thinking rationally); only the outcome measured by the performance measure matters.

(c) Rationality of the vacuum agent: scores over T = 8 steps

Simple reflex agent X (AIMA Fig. 2.8): if Dirty then Suck, else Right in A and Left in B. Agent Y keeps the set of squares known to be clean; if the square is Dirty it sucks, else if both squares are known clean it plays NoOp, else it moves to the other square. The agent starts in A, acts first, and the score (+1 per clean square, −1 per move) is added after each action.

Initial dirt (A, B)	Score of X (simple reflex)	Score of Y (reflex with state)
A clean, B clean	8	15
A clean, B dirty	8	14
A dirty, B clean	9	15
A dirty, B dirty	8	13
Average over the 4 configurations	8.25	14.25

Trace for the hardest configuration (both dirty). Step score = clean squares − move penalty.

Step	X: percept	X: action	X: clean	X: score	Y: percept	Y: action	Y: clean	Y: score
1	[A,Dirty]	Suck	1	+1	[A,Dirty]	Suck	1	+1
2	[A,Clean]	Right	1	0	[A,Clean]	Right	1	0
3	[B,Dirty]	Suck	2	+2	[B,Dirty]	Suck	2	+2
4	[B,Clean]	Left	2	+1	[B,Clean]	NoOp	2	+2
5	[A,Clean]	Right	2	+1	[B,Clean]	NoOp	2	+2
6	[B,Clean]	Left	2	+1	[B,Clean]	NoOp	2	+2
7	[A,Clean]	Right	2	+1	[B,Clean]	NoOp	2	+2
8	[B,Clean]	Left	2	+1	[B,Clean]	NoOp	2	+2
Total				**8**				**13**

Interpretation: with a penalty for moving, X is **not rational**: after both squares are clean it keeps shuttling and loses 1 point per step (e.g. 8 against 13). Y is rational because it uses its percept history (the internal state) to stop moving, and scores 5 to 7 points more than X in every configuration (average 14.25 against 8.25).

(d) Agent structure and table-driven size

Choice: goal-based agent. A reflex agent cannot choose among routes because the best action depends on the goal and on future consequences; a goal-based agent searches over the road map from its current city to the goal city. (A utility-based agent would be needed only if several conflicting criteria had to be traded off.) A table-driven design needs one entry per percept sequence of length 1 to 10, with 20 possible percepts:

$\Sigma_{t=1..10}\ 20^t = (20^{11} - 20) / 19 =$ **$10{,}778{,}947{,}368{,}420 \approx 1.08 \times 10^{13}$** entries (the last level alone is $20^{10} = 10{,}240{,}000{,}000{,}000$). This is far beyond any storage (10 terabytes even at 1 byte per entry), whereas the goal-based agent needs only the adjacency list: 2 × 23 = 46 entries (66 values with the 20 cities), about 10^{11} times smaller, and it also generalises to any start and goal.

(e) Problem formulation

Component	Formulation
States	In(city) for the 20 cities of the map
Initial state	In(Arad)
Actions	ACTIONS(In(x)) = {Go(y) : there is a road x–y}; e.g. ACTIONS(In(Arad)) = {Go(Sibiu), Go(Timisoara), Go(Zerind)}
Transition model	RESULT(In(x), Go(y)) = In(y)
Goal test	In(Bucharest)
Path cost	Sum of step costs c(In(x), Go(y), In(y)) = road distance in km

Three (plus one) uninformed strategies

Strategy	Idea and kind	Selection criterion for investigation	Preprocessing
S1: Breadth-first search (iterative, FIFO queue)	Expand the shallowest node first; goal test at generation; finds the route with the fewest roads	Expected nodes $b + b^2 + \ldots + b^d$; memory $\Theta(b^d)$; optimal only for equal step costs	Adjacency list; explored set
S2: Uniform-cost search (iterative, priority queue)	Expand the lowest path cost g(n) first; goal test at removal; replaces a frontier entry when a cheaper path is found	Cost optimality for road kilometres: nodes up to $b^{1+\lfloor C^*/\varepsilon \rfloor}$; memory same order	Adjacency list with km; explored set; best-g table
S3: Iterative deepening (recursive depth-limited DFS)	Repeat depth-limited recursive DFS with limit 0, 1, 2, …; combines BFS completeness with DFS memory	Memory O(bd) for very large maps; time $O(b^d)$ with overhead $b/(b-1)$	Adjacency list; path-cycle check
Baseline: DFS (recursive)	Follow the first branch as deep as possible, then backtrack	Memory O(bm); no optimality; used to show what is lost	Adjacency list; explored set

```
BFS(start, goal)
 if goal(start): return [start]
 frontier <- queue [start]
 explored <- {}; parent[start] <- none
 while frontier not empty:
   s <- dequeue; add s to explored
   for (t, km) in adj[s] (alphabetical):
     if t not in explored and not in
        frontier:  parent[t] <- s
        if goal(t): return path(t)
        enqueue t
 return failure

DLS(s, goal, limit, path)
 if goal(s): return path + [s]
 if limit = 0: return CUTOFF
 cut <- false
 for (t, _) in adj[s]  if t not in path:
    r <- DLS(t, goal, limit-1, path+[s])
    if r = CUTOFF: cut <- true
    elif r != failure: return r
 return CUTOFF if cut else failure
```

```
UCS(start, goal)
 frontier <- priority queue {(0, start)}; best[start] <- 0
  while frontier not empty:
     (g, s) <- pop lowest g;  skip if stale
     if goal(s): return path(s), g       # test at removal
     add s to explored
     for (t, km) in adj[s]:
       if t in explored: continue
        if g+km < best[t]: best[t] <- g+km; parent[t] <- s
                                push (g+km, t)   # replace dearer entry
   return failure


IDS(start, goal)
 for limit = 0, 1, 2, ... :
    r <- DLS(start, goal, limit, [])
    if r != CUTOFF: return r

 DFS_recursive(s): visited <- visited + {s}
     if goal(s): return path
     for t in adj[s] not in visited: r <- DFS_recursive(t); if r: return r
```

Task 2: Determine the basic operation (2 marks)

(a) Basic operation: node expansion, i.e. removing a node from the frontier and generating its successors from the adjacency list (which includes the goal test and the frontier insertions of those successors). **(b) Where it occurs:** BFS: the `popleft` and the loop over `adj[s]`; UCS: the heap pop and the loop over `adj[s]` (each insertion costs an extra O(log |frontier|)); DLS / IDS: each recursive call that is not cut off; recursive DFS: each call. **(c) Why it is appropriate:** every other costly step (successor generation, goal tests, frontier insertions, explored-set lookups) happens a bounded number of times per expansion (at most b = maximum degree), so total time is proportional to the number of expansions; it is executed in the innermost loop and is independent of machine speed.

Operation counts on the Romania map (Arad to Bucharest, measured):

Algorithm	Expansions	Nodes generated	Peak frontier	Route found (km)
BFS	5	8	5	Arad-Sibiu-Fagaras-Bucharest (450)
UCS (graph search)	12	13	4	Arad-Sibiu-Rimnicu Vilcea-Pitesti-Bucharest (418)
DFS (recursive, graph search)	3	3	–	Arad-Sibiu-Fagaras-Bucharest (450)
IDS (4 iterations, limits 0 to 3)	8	18	8	Arad-Sibiu-Fagaras-Bucharest (450)

(d) Growth as the map grows from 20 to 20,000,000 cities (average branching factor b = 2 × 23 / 20 = 2.3 is kept; with an explored set the number of expansions is at most the number of states V):

Cities V	Adjacency entries (2.3 V) built once	BFS / UCS expansions per query (worst case ≤ V)	Typical depth d ≈ $\log_{2.3} V$	IDS memory O(bd) ≈ 2.3 d nodes	BFS / UCS memory (frontier ≤ V)
20	46	≤ 20	3.6	≈ 8	≤ 20
2,000	4,600	≤ 2,000	9.1	≈ 21	≤ 2,000
200,000	460,000	≤ 200,000	14.7	≈ 34	≤ 200,000
20,000,000	46,000,000	≤ 20,000,000	20.2	≈ 46	≤ 20,000,000

The measured expansions (5 to 12 for one query on 20 cities) already follow this: they grow linearly with V when an explored set is used, while memory for BFS/UCS grows with V but IDS stays tiny. IDS repeats work, by a factor b/(b − 1) = 1.77 relative to BFS for b = 2.3.

Task 3: Derive the time and space complexity (6 marks)

Parameters (AIMA 3.4): branching factor b, depth of the shallowest goal d, maximum depth m, optimal cost C*, minimum step cost ε. Tree with b children per node: level i has b^i nodes.

(a) Number of nodes generated, derived by summing the levels

- **BFS** (goal test at generation) generates all nodes of levels 1 to d: $N = b + b^2 + \dots + b^d = b\,(b^d - 1)/(b - 1) = \mathbf{O(b^d)}$. (With the goal test at expansion it is $N + b^{d+1} - b$, still $O(b^{d+1})$.)
- **UCS** expands every node whose path cost is below C*. Each step costs at least ε, so such nodes lie at depth ≤ □C*/ε□, and nodes at the next level are also generated: $N \le 1 + b + \dots + b^{1+\square C^*/\varepsilon\square} = \mathbf{O(b^{1+\square C^*/\varepsilon\square})}$.
- **DFS** may traverse the entire tree down to depth m: $N = 1 + b + \dots + b^m = (b^{m+1} - 1)/(b - 1) = \mathbf{O(b^m)}$. **Depth-limited** with limit l: $O(b^l)$.
- **IDS** generates the nodes of level i once in each of the iterations with limit ≥ i, that is (d − i + 1) times: $N(\text{IDS}) = d\cdot b + (d-1)\cdot b^2 + \dots + 1\cdot b^d = \Sigma_{i=1..d}\,(d - i + 1)\,b^i = \mathbf{O(b^d)}$.

(b) Time and memory expressions

Algorithm	Time (nodes generated)	Memory (frontier size)	Derivation of memory	Complete / optimal
BFS	$b + \dots + b^d = O(b^d)$	$O(b^d)$	All nodes of the deepest level (b^{d-1} to b^d) are kept	Yes / yes if equal step costs
UCS	$O(b^{1+\square C^*/\varepsilon\square})$	$O(b^{1+\square C^*/\varepsilon\square})$	Frontier holds all nodes at cost ≈ C*	Yes (ε > 0) / yes
DFS	$O(b^m)$	O(bm)	Path of m nodes plus b − 1 siblings at each level: bm + 1	No (infinite m) / no
Depth-limited (limit l)	$O(b^l)$	O(bl)	Same as DFS with m = l	No / no
IDS	$\Sigma\,(d - i + 1)\,b^i = O(b^d)$	O(bd)	Only the current depth-limited path	Yes / yes if equal step costs

(c) Preprocessing cost and overall cost

Building the adjacency list inserts 2E entries for E roads and V cities: **O(V + E)** time and space (66 operations here: 20 cities + 46 entries), plus sorting each neighbour list alphabetically: O(Σ deg log deg) = O(E log b). The explored set costs O(1) expected per lookup and O(V) space. Overall cost for one query = preprocessing + search = O(V + E) + O(search). The preprocessing is done once and shared by all queries: for the 380 ordered city pairs it is 66 operations in total against an average of about 10 expansions per query, i.e. it is amortised to under 0.2 operations per query. With an explored set the search itself is bounded by O(V + E) (O((V + E) log V) for UCS), which is why graph search stays practical for 20,000,000 cities, whereas tree-search bounds such as b^d grow without limit when the map has cycles.

(d) Numerical check on the Romania map

b = 2 × 23/20 = 2.3, C* = 418 km, ε = 70 km, so □C*/ε□ = □5.97□ = 5 and the uniform-cost bound is $b^{1+5} = 2.3^6 = \mathbf{148.0}$ nodes. Measured with the implementation (Arad to Bucharest):

Quantity	Value	Compared with the bound 148
UCS expansions with explored set (graph search)	12 (13 nodes generated)	8.1% of the bound
UCS expansions without explored set (tree search)	52	35.1% of the bound
Bound $b^{1+\lfloor C^*/\varepsilon \rfloor}$	148.0	–

Both measured values are below the bound, as an upper bound must be. The bound is loose because ε = 70 km is the smallest road, most roads are longer (so fewer levels fit under C* = 418 km), and the explored set removes repeated cities.

Task 4: Best-, average- and worst-case behaviour (4 marks)

Inputs used: ordered city pairs of the Romania map (380 pairs; measured over all of them) and goals at depth d of the infinite binary tree (state k has successors 2k and 2k + 1).

Romania map (graph search, measured over all 380 ordered pairs)

Algorithm	Best case: condition → expansions	Average over 380 pairs	Worst case: condition → expansions
BFS	Goal is a neighbour of the start (46 such ordered pairs, e.g. Arad → Sibiu) → **1**	**7.65**	Goal found only when the last frontier layer is expanded, e.g. Zerind → Neamt (8 roads) → **19** (all other cities)
UCS	Goal is the cheapest neighbour (e.g. Arad → Zerind, 75 km) → **1**	**10.0**	Goal is the most expensive node to reach, e.g. Zerind → Neamt → **19**. Without an explored set the cheap-edge detours inflate this to up to **4,538** expansions (average 158.8)
DFS (recursive)	Goal on the first branch explored (Arad → Sibiu) → **1**	**10.0**	Goal is next to the start but on the last branch: Zerind → Oradea, DFS dives through Arad first → **19**
IDS	Goal is a neighbour → **1**	**21.15**	Deep goal plus repeated work and cycles, e.g. Lugoj → Neamt (9 roads) → **177**

Why 10.0 for UCS and DFS: for a given start, the 19 possible goals occupy the ranks 1 to 19 in the cost order (UCS) or visiting order (DFS), so the average number of expansions is (1 + 19)/2 = 10. BFS averages less (7.65) because it tests the goal when a node is generated, one step earlier.

Binary tree (b = 2), goal at depth d, nodes generated (examples d = 5 and d = 9, measured)

Algorithm	Best case: goal = leftmost node (2^d)	Average (5 random goals, d = 5 / 9)	Worst case: goal = rightmost node ($2^{d+1} - 1$)	Why
BFS	31 / 511	41.8 / 633.6	62 / 1,022 = $2^{d+1} - 2$	Best: first node of the last level; worst: last node, so the whole tree to depth d is generated
DLS (limit d)	10 / 18 = b·d	26.4 / 252.4	62 / 1,022	Best: goal on the first branch, only b·d nodes; worst: all nodes visited
IDS	62 / 1,022	78.4 / 1,256.4	114 / 2,026 = $\Sigma(d - i + 1)2^i$	Earlier iterations are repeated; worst case is exactly N(IDS)
UCS (goal at removal)	62 / 1,022	83.6 / 1,267.2	124 / 2,044	Expands one extra layer because the test is at removal: about 2 × BFS

Derivation of the input conditions. BFS generates the full tree above the goal level plus its j-th node at level d, so nodes = $(2^d - 2)$ + j with j between 1 and 2^d: best j = 1, worst j = 2^d, average ≈ 1.5 × 2^d. DFS follows the leftmost branch first, so its cost is b·d when the goal is on that branch and the whole tree ($2^{d+1} - 2$) when it is the last leaf; the average is about 2^d. UCS with unit costs behaves like BFS but expands every node of cost less than d and then reaches the goal at removal, hence about twice as many generated nodes. When edge costs differ (Romania), the worst case for UCS is a goal reached only after many cheap-edge detours.

Task 5: Asymptotic analysis (3 marks)

(a) N(BFS) and N(IDS) are both $\Theta(b^d)$

BFS. $N(BFS) = b + b^2 + \dots + b^d = b^d\,(1 + 1/b + 1/b^2 + \dots + 1/b^{d-1})$. Lower bound: $N \geq b^d$ (the last term alone) so $N = \Omega(b^d)$. Upper bound: the bracket is a geometric series $\leq 1/(1 - 1/b) = b/(b - 1)$, so $N \leq (b/(b - 1))\, b^d = O(b^d)$ (for $b \geq 2$ the constant is at most 2). Hence **$b^d \leq N(BFS) \leq (b/(b-1))\, b^d$, i.e. $\Theta(b^d)$**.

IDS. Put k = d − i: $N(IDS) = \Sigma_{k=0..d-1}\,(k + 1)\, b^{d-k} = b^d\, \Sigma_{k=0..d-1}\,(k + 1)\, x^k$ with x = 1/b. Lower bound: the k = 0 term gives $N \geq b^d$. Upper bound: $\Sigma_{k\geq 0}\,(k + 1)\, x^k = 1/(1 - x)^2$ (differentiate the geometric series), so $N \leq (b/(b - 1))^2\, b^d$ (for $b \geq 2$ at most 4 b^d). Hence **$b^d \leq N(IDS) \leq (b/(b-1))^2\, b^d$, i.e. $\Theta(b^d)$**, the same order as BFS.

Memory: BFS $\Theta(b^d)$; UCS $\Theta(b^{1+\lfloor C^*/\varepsilon \rfloor})$; DFS $\Theta(bm) = O(bm)$; IDS $\Theta(bd)$.

(b) Numerical check for b = 10, d = 5

N(BFS) = 10 + 100 + 1,000 + 10,000 + 100,000 = **111,110**. N(IDS) = 5·10 + 4·100 + 3·1,000 + 2·10,000 + 1·100,000 = 50 + 400 + 3,000 + 20,000 + 100,000 = **123,450**. Both agree with AIMA 3.4.5 (123,450 against 111,110). Excess = 123,450 − 111,110 = 12,340, i.e. **11.1%** more nodes than BFS (12,340 / 111,110 = 0.11106).

(c) Value of b for an iterative-deepening overhead below 10%

For large d, $N(BFS) \approx b^d \cdot b/(b-1)$ and $N(IDS) \approx b^d \cdot (b/(b-1))^2$, so $N(IDS)/N(BFS) \to b/(b-1)$ and the overhead $\to 1/(b-1)$. The overhead is below 10% when $1/(b-1) < 0.1$, i.e. **b > 11** (b = 11 gives exactly 10%). For the finite case d = 5 solving the exact formula numerically gives $b \approx 10.997$, so in practice **b ≥ 11**.

b	2	3	5	10	11	12	20
Overhead, d = 5 (exact)	83.9%	47.9%	24.8%	11.1%	10.0%	9.1%	5.3%
Limit 1/(b − 1)	100%	50%	25%	11.1%	10%	9.1%	5.3%

Task 6: Recurrence relations (3 marks)

(a) Recurrences for the nodes generated (root not counted)

Quantity	Recurrence	Base case
N(d): BFS to depth d	$N(d) = N(d-1) + b^d$	N(0) = 0
D(l): one depth-limited search, limit l	$D(l) = b \cdot (1 + D(l-1))$	D(0) = 0
I(d): iterative deepening, total to depth d	$I(d) = I(d-1) + D(d)$	I(0) = 0
S(m): nodes stored by DFS	$S(m) = S(m-1) + b$	S(0) = 1 (root)

(b) Components of each recurrence

Recurrence	Recursive part	Work added per level / step	Base case meaning
N(d)	N(d − 1): all shallower levels	b^d: the new deepest level	Depth 0: only the root, nothing generated
D(l)	D(l − 1): one subtree with limit l − 1	b children, each counted as 1 node plus its subtree D(l − 1), so $b + b \cdot D(l-1)$	Limit 0: the root is tested but not expanded
I(d)	I(d − 1): all earlier iterations	D(d): the new iteration with limit d (repeats everything above)	No iteration yet
S(m)	S(m − 1): storage for the shallower path	b: all children of the node expanded at this level are held	Only the root is stored

(c) Recurrence for the nodes stored by depth-first search

When DFS expands a node on level k − 1 it generates and stores its b children (one continues the path, b − 1 wait as siblings). Hence $S(m) = S(m-1) + b$ with S(0) = 1, which counts the memory after m expansions along one path (the path plus the unexplored siblings).

Task 7: Solve the recurrences (4 marks)

(a) Closed forms by repeated substitution / summation

N(d). $N(d) = N(d-1) + b^d = N(d-2) + b^{d-1} + b^d = \dots = N(0) + b + b^2 + \dots + b^d =$ **$b\,(b^d - 1)/(b-1)$**.

D(l). $D(l) = b + b \cdot D(l-1) = b + b^2 + b^2 \cdot D(l-2) = \dots = b + b^2 + \dots + b^l + b^l \cdot D(0) =$ **$b\,(b^l - 1)/(b-1)$**, the same as N(l): a depth-limited search generates exactly the nodes down to level l.

I(d). $I(d) = \Sigma_{l=1..d}\, D(l) = (b/(b-1))\, \Sigma_{l=1..d}\, (b^l - 1) = (b/(b-1))\, [\, b\,(b^d - 1)/(b-1) - d\,] =$ **$b\,[\, b\,(b^d - 1) - d\,(b-1)\,]\,/\,(b-1)^2$**.

S(m). $S(m) = S(0) + m \cdot b =$ **$b \cdot m + 1$**.

(b) Verification against direct evaluation

Case	Direct sum	Recurrence (code)	Closed form	Result
b = 10, d = 5: N	111,110	111,110	10 · 99,999 / 9 = 111,110	match
b = 10, d = 5: D(5)	111,110	111,110	same as N	match
b = 10, d = 5: I	123,450	123,450	10 [10 · 99,999 − 5 · 9] / 81 = 9,999,450 / 81 = 123,450	match
b = 2, d = 10: N	2,046	2,046	2 · 1,023 / 1 = 2,046	match
b = 2, d = 10: D(10)	2,046	2,046	same as N	match
b = 2, d = 10: I	4,072	4,072	2 [2 · 1,023 − 10] / 1 = 4,072	match

(c) Interpretation: when is the repeated work of iterative deepening acceptable?

The overhead $I(d)/N(d) - 1$ tends to $1/(b-1)$: **11.1% for b = 10** (12,340 extra nodes of 111,110) and **99.0% for b = 2, d = 10** (4,072 against 2,046, a factor 1.99). The repeated work is small and clearly acceptable when the branching factor is large ($b \geq 11$ gives ≤ 10%), because the last level dominates. Even for b = 2 it is only a constant factor of about 2 in time, while memory drops from $\Theta(b^d)$ (2,046 nodes) to $\Theta(bd)$ ($2 \cdot 10 + 1 = 21$ nodes). So IDS is acceptable whenever memory, not time, is the limiting resource and the depth of the goal is unknown.

Task 8: Implement and test the algorithms (4 marks)

Source code (Python 3). It is one module (`core.py`) with BFS, UCS, depth-limited and iterative-deepening DFS, a recursive DFS, the Romania map, the binary-tree successor function and the missionaries-and-cannibals formulation. Search code is generic: a problem is defined only by `succ(state)` and a goal test.

```
"""Uninformed search library: BFS, DFS (recursive), UCS, DLS, IDS on generic problems."""
import heapq
from collections import deque

# ---------- Romania road map (AIMA Fig. 3.2): 20 cities, 23 two-way roads ----------
ROADS = [("Oradea","Zerind",71),("Zerind","Arad",75),("Arad","Timisoara",118),("Arad","Sibiu",140),
 ("Oradea","Sibiu",151),("Sibiu","Fagaras",99),("Sibiu","Rimnicu Vilcea",80),("Timisoara","Lugoj",111),
 ("Lugoj","Mehadia",70),("Mehadia","Drobeta",75),("Drobeta","Craiova",120),("Craiova","Rimnicu Vilcea",146),
 ("Craiova","Pitesti",138),("Rimnicu Vilcea","Pitesti",97),("Pitesti","Bucharest",101),("Fagaras","Bucharest",211),
 ("Bucharest","Giurgiu",90),("Bucharest","Urziceni",85),("Urziceni","Hirsova",98),("Hirsova","Eforie",86),
 ("Urziceni","Vaslui",142),("Vaslui","Iasi",92),("Iasi","Neamt",87)]

def build_adjacency(roads):
    """Preprocessing: adjacency list, neighbours sorted alphabetically. O(V + E)."""
    adj = {}
    for a, b, km in roads:
        adj.setdefault(a, []).append((b, km))
        adj.setdefault(b, []).append((a, km))
    for city in adj:
        adj[city].sort()
    return adj

class Stats:
    def __init__(self):
        self.expanded = 0; self.generated = 0; self.peak = 0
        self.order = []          # order in which states are visited

def _path(parent, s):
    p = []
    while s is not None:
        p.append(s); s = parent[s]
    return p[::-1]

# ---------- 1. Breadth-first search (iterative, FIFO queue, explored set, goal test at generation) ----------
def bfs(start, goal, succ):
    st = Stats(); st.order.append(start)
    if goal(start): return [start], st
    parent = {start: None}; frontier = deque([start]); explored = set(); st.peak = 1
    while frontier:
        s = frontier.popleft(); explored.add(s); st.expanded += 1
        for t, _ in succ(s):
            if t not in explored and t not in parent:
                parent[t] = s; st.generated += 1; st.order.append(t)
                if goal(t): return _path(parent, t), st
                frontier.append(t)
        st.peak = max(st.peak, len(frontier))
    return None, st

# ---------- 2. Uniform-cost search (iterative, priority queue, goal test at removal) ----------
def ucs(start, goal, succ, explored_set=True):
    st = Stats(); tie = 0
    heap = [(0, tie, start)]; best = {start: 0}; parent = {start: None}; explored = set(); st.peak = 1
    while heap:
        g, _, s = heapq.heappop(heap)
        if explored_set and s in explored: continue
        if explored_set and g > best.get(s, g): continue
        st.order.append(s)
        if goal(s):
            path = [];
            if explored_set: return _path(parent, s), g, st
            return None, g, st
        explored.add(s); st.expanded += 1
        for t, c in succ(s):
            ng = g + c
            if explored_set:
                if t in explored or ng >= best.get(t, float("inf")): continue
                best[t] = ng; parent[t] = s
            tie += 1; heapq.heappush(heap, (ng, tie, t)); st.generated += 1
        st.peak = max(st.peak, (len(best) - len(explored)) if explored_set else len(heap))
    return None, None, st

# ---------- 3. Depth-limited DFS (recursive) and iterative deepening ----------
CUTOFF = "cutoff"
def dls(start, goal, succ, limit, st=None, path=None, pending=None):
    """Recursive depth-limited search; avoids cycles along the current path.
       pending = generated-but-unvisited nodes (the DFS 'frontier')."""
    if st is None: st = Stats(); pending = [0]; path = []
    st.order.append(start)
    if goal(start): return path + [start], st
    if limit == 0: return CUTOFF, st
    st.expanded += 1; cut = False
    kids = [t for t, _ in succ(start) if t not in path]
    st.generated += len(kids); pending[0] += len(kids); st.peak = max(st.peak, pending[0] + len(path) + 1)
    for t in kids:
        pending[0] -= 1
        r, _ = dls(t, goal, succ, limit - 1, st, path + [start], pending)
        if r == CUTOFF: cut = True
        elif r is not None: return r, st
    return (CUTOFF if cut else None), st

def ids(start, goal, succ, max_depth=10**6):
    total = Stats(); iterations = 0
```

```
    for lim in range(max_depth + 1):
        iterations += 1
        r, st = dls(start, goal, succ, lim)
        total.expanded += st.expanded; total.generated += st.generated
        total.peak = max(total.peak, st.peak); total.order += st.order
        if r != CUTOFF: return r, lim, iterations, total
    return None, None, iterations, total

# ---------- 4. Recursive depth-first graph search (global explored set; complete on finite graphs) ----------
def dfs_graph(start, goal, succ):
    st = Stats(); seen = set()
    def rec(s, path):
        seen.add(s); st.order.append(s)
        if goal(s): return path + [s]
        st.expanded += 1
        for t, _ in succ(s):
            if t not in seen:
                st.generated += 1
                r = rec(t, path + [s])
                if r: return r
        return None
    return rec(start, []), st

def romania_succ(adj): return lambda s: adj.get(s, [])
def path_cost(adj, path):
    d = {}
    for a, nb in adj.items():
        for b, k in nb: d[(a, b)] = k
    return sum(d[(path[i], path[i+1])] for i in range(len(path) - 1))

def tree_succ(k): return [(2*k, 1), (2*k + 1, 1)]

# ---------- Missionaries and cannibals (state = (m, c, boat), m and c on the LEFT bank) ----------
def mc_legal(m, c):
    return (m == 0 or c <= m) and ((3 - m) == 0 or (3 - c) <= (3 - m))
MC_LOADS = [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]
def mc_succ(s):
    m, c, b = s; out = []
    for dm, dc in MC_LOADS:
        if b == 0:  nm, nc, nb = m - dm, c - dc, 1
        else:       nm, nc, nb = m + dm, c + dc, 0
        if 0 <= nm <= 3 and 0 <= nc <= 3 and mc_legal(nm, nc): out.append(((nm, nc, nb), 1))
    return out
```

Test driver (excerpt; the full run uses `assert` for the expected behaviour, all assertions passed):

```
adj = build_adjacency(ROADS); succ = romania_succ(adj)
p, c, st = ucs('Arad', G('Bucharest'), succ);  assert p == ['Arad','Sibiu','Rimnicu Vilcea','Pitesti','Bucharest'] and c == 418
p, st    = bfs('Arad', G('Bucharest'), succ);  assert p == ['Arad','Sibiu','Fagaras','Bucharest'] and path_cost(adj, p) == 450
p, st = bfs(1, G(11), tree_succ);   assert st.order == list(range(1, 12))
r, st = dls(1, G(11), tree_succ, 3); assert st.order == [1, 2, 4, 8, 9, 5, 10, 11]
r, lim, it, st = ids(1, G(11), tree_succ); assert it == 4
p, st = bfs((3, 3, 0), G((0, 0, 1)), mc_succ);  assert len(p) - 1 == 11
```

Test input, expected behaviour, actual output

Test	Input	Expected behaviour	Actual output	Metrics (expanded / generated / peak)
(a) UCS, Romania	Arad → Bucharest	Arad-Sibiu-Rimnicu Vilcea-Pitesti-Bucharest, 418 km	Same route, 418 km □	12 / 13 / 4
(a) BFS, Romania	Arad → Bucharest	Fewest roads: Arad-Sibiu-Fagaras-Bucharest, 450 km	Same route, 3 roads, 450 km □	5 / 8 / 5
(a) DFS and IDS, Romania	Arad → Bucharest	Some route (not nec. cheapest)	Both return Arad-Sibiu-Fagaras-Bucharest, 450 km; IDS limit 3 in 4 iterations	DFS 3 / 3; IDS 8 / 18
(b) BFS, binary tree	goal 11	Visits states 1 to 11 in order	[1,2,3,4,5,6,7,8,9,10,11] □	5 / 10 / 5
(b) DLS limit 3	goal 11	Visits 1,2,4,8,9,5,10,11	[1,2,4,8,9,5,10,11], path 1-2-5-11 □	–
(b) IDS	goal 11	Goal found in the 4th iteration	Found at limit 3, iteration 4 □	–
(c) Missionaries and cannibals	state (m, c, boat), start (3,3,L)	11 crossings; legal and reachable states	11 crossings (BFS: 13 expansions; IDS: 11 in iteration 12). Legal states: **20** (10 legal (m,c) pairs × 2 boat sides); reachable from the start: **16** □	–

Optimal missionaries-and-cannibals solution found (m, c, boat; boat L = left bank): (3,3,L) → (3,1,R) → (3,2,L) → (3,0,R) → (3,1,L) → (1,1,R) → (2,2,L) → (0,2,R) → (0,3,L) → (0,1,R) → (1,1,L) → (0,0,R): 11 crossings.

(d) Boundary conditions

Test	Expected	Actual output
Start equals goal (Arad → Arad)	Path [Arad], cost 0, no expansion	BFS: [Arad], 0 expanded; UCS: [Arad], cost 0 □
Unreachable goal (added isolated city Atlantis)	Failure after exploring the reachable part, no infinite loop	BFS, UCS and DFS return None after 20 expansions; IDS (max depth 25) returns None after 1,295 expansions □

Test	Expected	Actual output
City with a single road (Neamt, only road Iasi–Neamt 87 km)	Works from and to the dead end	Neamt → Bucharest: Neamt-Iasi-Vaslui-Urziceni-Bucharest, 406 km; reverse route identical, 406 km □
Repeated states (cycle Oradea–Zerind–Arad–Sibiu)	Explored set or path check prevents loops	UCS Oradea → Bucharest: 429 km with 10 expansions using the explored set, against 43 expansions without it (same cost). A recursive DFS with no repeated-state check raises RecursionError on the cycle, whereas DLS with the path check terminates □

Task 9: Record experimental performance (3 marks)

Predictions (written before running the experiments)

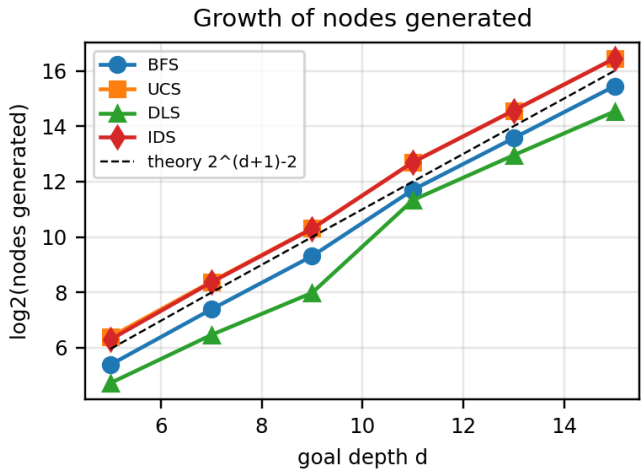
- BFS generates between $2^d - 1$ and $2^{d+1} - 2$ nodes for a goal at depth d; averaged over random goals ≈ 1.5×2^d (Task 4). Peak frontier ≈ 2^{d-1} to 2^d.
- UCS with unit costs and the goal test at removal generates ≈ 2 × the BFS count, with the same peak frontier.
- Depth-limited DFS (limit d) visits about half of the tree for a random goal: ≈ 2^d generated; peak frontier $b \cdot d + 1 = 2d + 1$.
- IDS generates ≈ $b/(b - 1) = 2 \times$ the BFS count and has peak frontier $2d + 1$.
- Romania: UCS finds the cheapest route in 100% of the pairs; BFS and IDS return fewest-road routes (cheapest in under 100% of the pairs); DFS is the worst on cost.

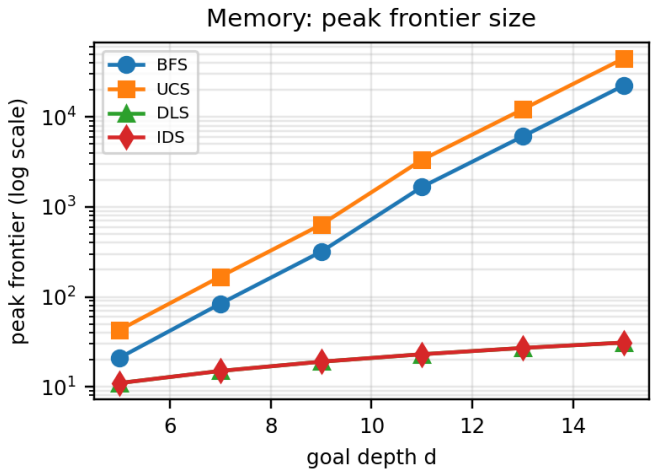
(a) Binary-tree space: goals at depths 5 to 15

Method: for each depth d, 5 goals drawn at random from the 2^d nodes of that level (`random.seed(42)`); table entries are the mean of the 5 runs, shown as nodes generated / nodes expanded / peak frontier. DFS without a depth limit is not run because it is incomplete on an infinite tree; depth-limited search with limit d is used instead.

d	Goals (seed 42)	BFS	UCS	DLS (limit d)	IDS
5	39, 33, 49, 47, 46	41.8 / 21.0 / 21.0	83.6 / 41.8 / 42.8	26.4 / 13.2 / 11.0	78.4 / 39.2 / 11.0
7	163, 154, 150, 236, 136	166.8 / 83.8 / 83.8	333.6 / 166.8 / 167.8	88.0 / 44.0 / 15.0	328.0 / 164.0 / 15.0
9	542, 607, 735, 750, 539	633.6 / 317.0 / 317.0	1,267.2 / 633.6 / 634.6	252.4 / 126.2 / 19.0	1,256.4 / 628.2 / 19.0
11	2862, 3766, 2950, 3887, 3187	3,329.4 / 1,665.0 / 1,665.0	6,658.8 / 3,329.4 / 3,330.4	2,574.0 / 1,287.0 / 23.0	6,646.0 / 3,323.0 / 23.0
13	8298, 10807, 15116, 13766, 12744	12,145.2 / 6,073.0 / 6,073.0	24,290.4 / 12,145.2 / 12,146.2	7,922.8 / 3,961.4 / 27.0	24,278.8 / 12,139.4 / 27.0
15	42957, 46878, 54827, 39466, 38846	44,593.8 / 22,297.2 / 22,297.2	89,187.6 / 44,593.8 / 44,594.8	23,666.8 / 11,833.4 / 31.0	89,170.8 / 44,585.4 / 31.0

Worst case (goal = rightmost node of level d), generated nodes: BFS and DLS 62, 254, 1,022, 4,094, 16,382, 65,534 = $2^{d+1} - 2$; IDS 114, 494, 2,026, 8,166, 32,738, 131,038 = $\Sigma\ (d - i + 1)\ 2^i$; UCS exactly twice BFS (124 … 131,068). Peak frontier for DLS and IDS: 11, 15, 19, 23, 27, 31 = $2d + 1$.

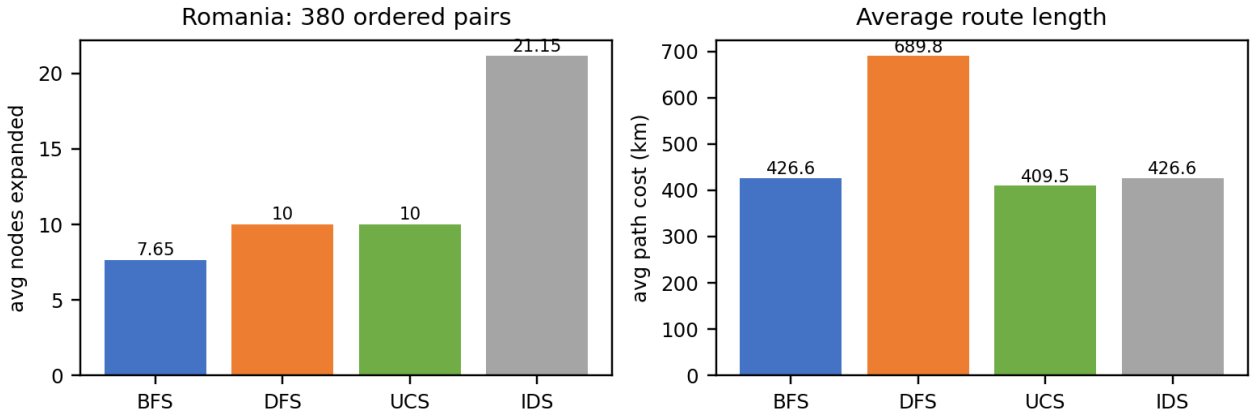




(b) Romania map: all 380 ordered city pairs

Algorithm	Average nodes expanded	Average nodes generated	Average path cost (km)	Share of pairs with the cheapest route
BFS (fewest roads)	7.65	10	426.6	71.1%
DFS (recursive, graph search)	10	10	689.8	35.3%

Algorithm	Average nodes expanded	Average nodes generated	Average path cost (km)	Share of pairs with the cheapest route
UCS	10	12.36	409.5	100.0%
IDS (path-cycle check)	21.15	35.24	426.6	71.1%



(c) Reproducibility: the algorithms are deterministic (alphabetical neighbour order), so the only randomness is the goal choice in (a), seeded with 42 and repeated 5 times per depth; running the code again reproduces every number in the tables.

Task 10: Compare theory with experimental behaviour (4 marks)

(a) Growth on the binary tree compared with b^d and the predictions

Algorithm	Measured slope of $\log_2$(nodes generated) vs d	Implied growth factor per level	Predicted (b = 2)	Comment
BFS	1.018	2.025	1.000 / 2	Within 1.3% of theory
UCS	1.018	2.025	1.000 / 2	Same shape, constant factor 2
DLS (limit d)	1.027	2.037	1.000 / 2	Within 1.9% (5-sample noise)
IDS	1.026	2.036	1.000 / 2	Within 1.8%

The measured growth is essentially 2^d, confirming $\Theta(b^d)$. The average BFS count divided by 2^d is 1.31, 1.30, 1.24, 1.63, 1.48 and 1.36 for d = 5 … 15 against the predicted 1.5, so predictions hold within the sampling noise of 5 random goals (the worst-case runs match the formulas exactly: $2^{d+1} - 2$ for BFS and DLS, $\Sigma\ (d - i + 1)\ 2^i$ for IDS). The predicted peak memory also matched: 2d + 1 for DLS and IDS (11 to 31), growing linearly, against exponential growth (about 2^{d-1}) for BFS and UCS.

(b) Discrepancies and their cause

- **DFS on the Romania map explores far fewer nodes than b^m.** With b = 2.3 and m up to 19 the tree formula gives $2.3^{19} \approx 7.5 \times 10^6$ nodes, but the measured maximum is 19 expansions (average 10). Cause: the map has only 20 states; the recursive DFS with an explored set never re-expands a city, while the b^m formula counts every path as a separate node, as in a tree.
- **UCS bound.** The bound 148 for Arad → Bucharest compares with 12 expansions (graph search) and 52 (tree search): the same cause (explored set) plus the fact that ε = 70 km is smaller than most roads.
- **BFS** averages 7.65 expansions against $b^d \approx 22$ for the average depth of 3.72 roads: the explored set and the goal test at generation again.
- **DFS route quality:** 689.8 km on average against 409.5 km for UCS; DFS found the cheapest route in only 35.3% of the pairs, since it takes the first alphabetical branch and does not compare routes.

(c) Measured iterative-deepening overhead against the Task 5 value

Binary tree (b = 2): IDS / BFS generated nodes = 131,038 / 65,534 = 1.9995 at d = 15 (overhead 99.95%) and 2.00 on average over the random goals, matching the Task 5 limit 1/(b − 1) = 100% and the closed form (99.02% at d = 10, Task 7). Romania: IDS generates 35.24 nodes per query against 10 for BFS, a factor 3.5 (much larger than b/(b − 1) = 1.77 for b = 2.3). The extra overhead comes from cycles of the map: IDS checks only the current path, not an explored set, so the same city is regenerated through different paths in every iteration. The theory for trees is therefore a lower bound for graphs.

Task 11: Recommend and defend the final design (4 marks)

Recommendation: uniform-cost search with an adjacency list and an explored set (graph search), for the Romania map and, with a priority queue on the adjacency lists, for 20,000,000 cities. Road distances are unequal, so only UCS returns the cheapest route (100% of the 380 pairs; BFS and IDS 71.1%, DFS 35.3%), and its cost $O((V + E) \log V)$ with an explored set remains manageable for 2×10^7 cities (about 10^9 basic operations; the frontier may reach V nodes, a few gigabytes, which is the main limit).

(a) Conditions under which the recommendation changes (compare AIMA Fig. 3.21)

Condition	Best choice	Reason
All edge costs equal (or hop count is the cost)	BFS	Optimal, simpler and faster than UCS (no heap): average 7.65 against 10 expansions
Memory is the limiting resource and costs are equal	IDS	O(bd): about 2.3 × 20 ≈ 46 nodes at 2 × 10^7 cities against up to 2 × 10^7 for BFS/UCS; time overhead b/(b − 1) = 1.77
Memory limited and costs unequal	Iterative lengthening / bounded-cost search, or UCS on a bounded region	Plain IDS ignores cost; UCS needs the frontier
Any route will do, depth bounded, tiny memory	Depth-limited DFS	O(bm) memory but not optimal (689.8 km on average)
Very small ε relative to C*	Avoid plain UCS tree search	$b^{1+\lfloor C^*/\varepsilon \rfloor}$ grows; use the explored set
Infinite or very deep space	BFS, UCS or IDS (never plain DFS)	DFS is incomplete when m is infinite or cycles are not checked

(b) Evaluation against an alternative using my own measurements

Measure (Romania)	UCS (recommended)	BFS (alternative)	IDS	DFS
Arad → Bucharest route	418 km (12 expansions)	450 km (5 expansions)	450 km (8 expansions, 4 iterations)	450 km (3 expansions)
Average route over 380 pairs	409.5 km	426.6 km (4.2% longer)	426.6 km	689.8 km (68% longer)
Pairs with the cheapest route	100%	71.1%	71.1%	35.3%
Average nodes expanded	10.0	7.65	21.15	10.0
Worst-case expansions	19	19	177	19

BFS is 24% cheaper in expansions (7.65 against 10.0) but returns a longer route in 28.9% of the pairs (for example 32 km longer on Arad → Bucharest: 450 km against 418 km), so for a flight-deadline routing problem UCS is the better trade-off. The binary-tree measurements show the same pattern for the unit-cost case: BFS and UCS find the same depth, with UCS using about twice as many generated nodes only because of its removal-time goal test.

Task 12 (Day-4 design challenge): search with partial information (8 marks)

Physical states of the two-square vacuum world, written (agent, left square, right square): S1 (L,D,D), S2 (R,D,D), S3 (L,D,C), S4 (R,D,C), S5 (L,C,D), S6 (R,C,D), S7 (L,C,C), S8 (R,C,C). Goal: both squares clean (S7 or S8).

A. Impact analysis (2 marks)

(a) Assumptions that no longer hold. (1) *Full observability*: the percept no longer identifies the state (there is none), so the agent cannot know in which of the 8 states it is. (2) *Known start state*: the start can be any of the 8 states. (3) The solution can no longer be a path in physical-state space chosen with knowledge of the current state; it must be a fixed action sequence that works from every possible state (a conformant plan). Actions are still deterministic and the goal is unchanged.

Size of the spaces (n squares: physical states $n \cdot 2^n$, belief states $2^{(n \cdot 2^n)}$):

n	Physical states $n \cdot 2^n$	Possible belief states $2^{n \cdot 2^n}$
2	2 × 4 = 8	2^8 = 256
3	3 × 8 = 24	2^{24} = 16,777,216

(b) Belief states (computed by the code). Initial belief {S1, S2, S3, S4, S5, S6, S7, S8}. After **Right**: {S2, S4, S6, S8} (the agent is certainly in the right square, with unknown dirt). After **Right, Suck**: {S4, S8} (the right square is certainly clean). Knowledge about location and dirt is therefore part of the state; the agent gains certainty from its own actions.

B. Evaluation of the original design (2 marks)

(a) Can the original algorithm still be applied? Test: the original BFS (written for physical states) was run on the belief-state example with the physical successor function. It fails: `TypeError: 'frozenset' object is not subscriptable`, because a state is no longer a (position, dirt) tuple. What a "state" becomes: **a belief state, i.e. a set of physical states** (a frozenset). The algorithm itself (queue, explored set, goal test) can be kept; only the state, successor function (predict the set of results of the action) and goal test (every physical state in the set is a goal) change. With these changes the same `bfs` function returns a path of 4 actions (10 belief states generated, 9 expanded).

(b) Reachable belief states. Starting from the initial belief state and applying Left, Right and Suck repeatedly, the program finds **12** reachable belief states out of 256 (4.7%), which agrees with AIMA:

Size	Reachable belief states
8	{S1,…,S8} (initial)
4	{S1,S3,S5,S7}, {S2,S4,S6,S8}, {S4,S5,S7,S8}

Size	Reachable belief states
3	{S3,S5,S7}, {S4,S6,S8}
2	{S3,S7}, {S4,S8}, {S5,S7}, {S6,S8}
1	{S7}, {S8}

Judgement: the original design as written (search over physical states assuming a known start) is **not acceptable**: its plans may fail from other starting states. Worked example: the plan [Suck, Right, Suck] solves S1 (known start) but applied to the whole initial belief gives {S4, S5, S7, S8} → {S4, S6, S8} → {S4, S8}, and S4 = (R,D,C) still has a dirty left square. However the search method is reusable, and the reachable belief space is small (12 states).

C. Adaptation or redesign (2 marks)

(a) Revised strategy: MODIFY. Keep BFS and redefine the state as a belief state; actions are Left, Right, Suck; successor = set of results; goal = all physical states in the belief are clean; path cost 1 per action. Complexity: worst case $2^{n \cdot 2^n}$ belief states, but only reachable ones are generated (12 for n = 2, 66 for n = 3, 421 for n = 4). Running it from the initial belief state finds a shortest plan of 4 actions: with action order (Right, Left, Suck) BFS returns **[Right, Suck, Left, Suck]**, with order (Left, Right, Suck) it returns the equivalent mirror plan [Left, Suck, Right, Suck]; exhaustive enumeration shows these are the only two plans of length 4. Both end in belief state {S7}, so the revised search finds a plan equivalent to [Right, Suck, Left, Suck].

Option with a local sensor [location, dirt of the current square]: after each action the predicted belief state is split by the percept (belief update), and the plan becomes conditional (an AND–OR search). For n = 2 the initial percept splits the belief into {S1,S3}, {S2,S6}, {S5,S7}, {S4,S8}. The conditional plan is “Suck if dirty; move to the other square; Suck if dirty”, at most 3 actions, whatever the start.

(b) If the original solution were retained (physical plan from an assumed start), it would fail, as the worked example in B(b) shows (final belief {S4, S8} is not a goal).

D. Evidence-based defence (2 marks)

Decision: modify the design: keep BFS but search over belief states. **Mathematical argument:** without sensors a plan must work for every possible state, so it has to reduce uncertainty by actions (coercion). For n squares in a line the plan is: n − 1 times Left (puts the agent in the leftmost square whatever the start), then Suck, and (Right, Suck) repeated n − 1 times, which is (n − 1) + 1 + 2(n − 1) = **3n − 2** actions: 4, 7 and 10 for n = 2, 3, 4, exactly the lengths found by the search below. **Small experiment** (code below; sensorless = BFS over belief states; with sensor = AND–OR search for the minimum worst-case length of a conditional plan):

n squares	Physical states	Reachable belief states	Sensorless: plan length	Sensorless: belief states generated (BFS successors)	Sensorless: distinct beliefs visited	With sensor: worst-case plan length	With sensor: distinct beliefs visited
2	8	12	4	27	11	3	12
3	24	66	7	165	62	6	46
4	64	421	10	1,023	386	8	137

Interpretation: sensorless plan length equals 3n − 2 as derived. The number of reachable belief states grows by about 5.5× to 6.4× per extra square (12 → 66 → 421) but remains tiny compared with $2^{n \cdot 2^n}$ (256, 16.8 million, about 1.8×10^{19}), so searching only the reachable belief states is practical. A local sensor shortens the worst-case plan (3, 6, 8 against 4, 7, 10) and explores fewer belief states (12, 46, 137), at the price of conditional plans and sensor hardware; the sensorless plan is a single fixed sequence and still guarantees the goal from every state, so the redesigned solution is justified in both cases.

Code for the experiment (`belief.py`; it reuses the search code of Task 8):

```
"""Task 12: sensorless and local-sensor search in the n-square vacuum world (squares in a line)."""
import sys, itertools, time; sys.path.insert(0, '.')
from core import bfs
from collections import deque

def states(n): return [(p, m) for p in range(n) for m in range(2 ** n)]      # (position, dirt bitmask)
def step(s, a, n):
    p, m = s
    if a == 'Left':  return (max(0, p - 1), m)
    if a == 'Right': return (min(n - 1, p + 1), m)
    return (p, m & ~(1 << p))                                                 # Suck
ACTIONS = ['Left', 'Right', 'Suck']
def predict(b, a, n): return frozenset(step(s, a, n) for s in b)              # belief after an action
def percept(s): p, m = s; return (p, (m >> p) & 1)                            # [location, dirty?]
def split(b):                                                                 # belief update by percept
    d = {}
    for s in b: d.setdefault(percept(s), set()).add(s)
    return [frozenset(v) for v in d.values()]
def goal(b): return all(m == 0 for _, m in b)

def sensorless(n, order=('Right', 'Left', 'Suck')):
    init = frozenset(states(n)); parent = {init: None}; q = deque([init]); generated = 0
    if goal(init): return [], 0, 1
    while q:                                                                  # BFS over belief states
        b = q.popleft()
        for a in order:
            t = predict(b, a, n); generated += 1
            if t not in parent:
```

```
                parent[t] = (b, a)
                if goal(t):
                    plan = []; x = t
                    while parent[x]: x, a2 = parent[x][0], parent[x][1]; plan.append(a2)
                    return plan[::-1], generated, len(parent)
                q.append(t)
def all_shortest(n, length):
    init = frozenset(states(n)); out = []
    def rec(b, plan):
        if goal(b): out.append(plan); return
        if len(plan) == length: return
        for a in ACTIONS: rec(predict(b, a, n), plan + [a])
    rec(init, []); return [p for p in out if len(p) == length]
def reachable(n):
    init = frozenset(states(n)); seen = {init}; q = deque([init])
    while q:
        b = q.popleft()
        for a in ACTIONS:
            t = predict(b, a, n)
            if t not in seen: seen.add(t); q.append(t)
    return seen

def with_sensor(n):
    """AND-OR iterative deepening: minimal worst-case length of a conditional plan; returns (length, #distinct beliefs, #calls)."""
    memo = {}; calls = [0]; seen = set()
    def solve(b, depth):
        key = (b, depth)
        if key in memo: return memo[key]
        calls[0] += 1; seen.add(b)
        if goal(b): memo[key] = True; return True
        if depth == 0: memo[key] = False; return False
        res = False
        for a in ACTIONS:
            t = predict(b, a, n)
            if t == b: continue
            if all(solve(sub, depth - 1) for sub in split(t)): res = True; break
        memo[key] = res; return res
    init = frozenset(states(n)); D = 0
    while True:
        if all(solve(sub, D) for sub in split(init)): return D, len(seen), calls[0]
        D += 1
```

Output of the experiment (printed by the program):

```
n | physical | sensorless plan | length | generated | distinct beliefs visited | reachable beliefs | sensor worst-case | sensor beliefs | sensor calls
2 |    8 | [Right, Suck, Left, Suck]                                    |  4 |   27 |  11 |  12 | 3 |  12 |  35
3 |   24 | [Right, Right, Suck, Left, Suck, Left, Suck]                  |  7 |  165 |  62 |  66 | 6 |  46 | 181
4 |   64 | ... (10 actions)                                              | 10 | 1023 | 386 | 421 | 8 | 137 | 669
initial percept split n=2: ['{S1, S3}', '{S2, S6}', '{S5, S7}', '{S4, S8}']
plans, n=2, shortest length 4: [['Left','Suck','Right','Suck'], ['Right','Suck','Left','Suck']]
```